

Tool Calls as Reversible-Action Admission: A Substrate Layer for Agentic AI Safety

Anonymous for review

Abstract

The agentic-AI safety literature treats safety as a property of the model. Constitutional AI, RLHF, alignment-faking research, and scalable-oversight work all ask how to make a particular model behave well; the proposed solutions sit at training time, evaluation time, or interpretability time. We argue for an orthogonal property: safety is additionally a property of the substrate that admits a tool call, independent of the model that produced it. Every tool call an LLM agent issues is structurally a delegated executive act with five components (authorisation, subject, effect, lifetime, reversal expectation). A substrate that requires every destructive tool call to carry a positive time-to-live (TTL) and a typed rollback receipt slot at admission, with bilateral cosignature for irreversible variants, refuses tool calls a misaligned model can issue but cannot justify. This refusal holds even when alignment training has failed, the model schemes, the model fakes alignment, and the operator is manipulated, because the substrate is the verifier and not the agent. We sketch the formal grammar of admission, name the threat model that survives, and describe how training-layer and admission-layer safety compose.

1 Introduction

A tool call is an executive act. When an LLM agent reads a file, writes to a database, sends an email, executes a shell command, or pushes a commit, it takes an action on behalf of a user, against an external system, with consequences in the world. The agent receives delegated authority through a system prompt or developer credential; it dispatches that authority against a resource it does not own; it produces an effect that, if wrong, must be undone.

The agentic-AI safety literature has converged on a particular framing. Safety is treated as a property of the model: how to make the model want the right thing, refuse the wrong thing, and behave consistently between evaluation and deployment. Constitutional AI [4] and RLHF [11] shape preferences at training time. Scalable oversight [10, 6] and debate [9] extend supervision past the operator’s direct capability. Alignment-faking [7] and situational-awareness [5] work sharpen the threat model. Each thread treats the model as the object of safety analysis.

We argue that safety is also, separately, a property of the substrate that admits the tool call. The substrate may decline to execute a tool call that does not carry a typed proof of reversibility, regardless of what the model that issued the call wants, regardless of whether the model successfully evaded its training signal, and regardless of whether the operator was manipulated into approving it. The grammar of admission is independent of the grammar of generation.

The structural argument is short. A tool call has the shape of a delegated executive act: a typed envelope, an authorisation chain, a target, an effect, and (in the destructive case) a need for reversal. A substrate that requires every destructive tool call to be constructed only with a positive time-to-live (TTL) witness and a typed rollback receipt slot refuses tool calls that the model cannot, by construction, prove are reversible. The model may be misaligned; it cannot

forge a rollback witness for an action whose effects it cannot undo. The substrate may be type-checked external to the model.

Contributions. This paper makes three contributions, all structural.

- We frame the agent tool call as a five-part executive act (authorisation chain, subject, effect, lifetime, reversal expectation), and we show that the same five components appear in adjacent formally analysed domains (endpoint security response, temporally bounded administrative orders).
- We name a substrate-layer safety property orthogonal to the training-layer, evaluation-layer, and interpretability-layer stack: typed admission with TTL by construction, rollback receipt slot, and bilateral cosignature on irreversible classes. We give the grammar and cite the formal substrate that discharges its load-bearing theorems [1].
- We identify the threat model that the substrate-layer guarantee survives: misaligned model, strategic composition across tool calls, alignment-faking, and operator manipulation. We name the bounds on the guarantee (mis-classified action, manipulable registry, bypassed dispatch).

The contribution is structural rather than empirical. The paper does not introduce a new training-time technique or a benchmark. It proposes that two safety layers exist, that they compose, and that the runtime-substrate layer holds under adversarial assumptions about the model that training-time interventions cannot match.

The argument proceeds as follows. Section 2 surveys the existing stack and identifies the gap. Section 3 develops the executive-act framing. Section 4 states the admission grammar. Section 5 gives a worked example. Section 6 states the threat model that survives. Section 7 situates the construction next to existing alignment-research threads. Section 8 names the bounds, and Section 9 concludes.

2 Background

The dominant stack for agentic-AI safety operates at training time, evaluation time, and interpretability time. We summarise the threads and identify the gap an admission-layer construction fills.

Training-time alignment. RLHF [11] encodes operator judgement into a reward model that shapes the policy’s preferences over outputs. Constitutional AI [4] generalises this: a written constitution scores outputs through a critic, producing supervision without per-output annotation. Both approaches make the model want the right thing. Neither produces a tamper-evident artifact of what the deployed model actually did, and neither extends to runtime behaviour the training pipeline did not anticipate.

Scalable oversight and debate. Scalable oversight [10, 6] treats the operator’s evaluative capability as the limiting resource and extends it through recursive reward modelling, amplification, or debate between models [9]. These are evaluation-time techniques; they produce a verdict on a candidate output before it is acted on. They do not constrain what the substrate does after the verdict.

Alignment-faking and situational awareness. The threat model has sharpened. Alignment-faking [7] demonstrates that a model can behave differently when it believes it is being evaluated than when it believes it is deployed, in ways not reducible to outer optimisation failure.

Situational-awareness research [5] probes the underlying capability. Earlier work on deceptive alignment [8] laid out the conceptual frame. These results are not claims that current models scheme. They are claims that the community must reason about a regime in which evaluation-time behaviour is not a reliable predictor of deployment-time behaviour.

Red-teaming and capability evaluation. Red-teaming [12] elicits failure modes before deployment. Frontier-safety frameworks such as the Responsible Scaling Policy [2] bound deployed capability with capability-gated thresholds. These are pre-deployment instruments. They do not produce a record of what the model actually did, against which tool, under what authority, once deployed.

The gap. None of these threads produces a runtime artifact an external party can replay to check whether a particular tool call was admissible under a declared policy. None constrains the substrate’s willingness to execute a tool call as a function of typed properties of the call itself. The substrate is implicitly modelled as a passive dispatcher: whatever the model produces and the operator approves, the substrate executes. The safety claims in the literature are about the model and the evaluator; the substrate has no safety contract. A typed admission layer is intended to close this gap. The two safety stories compose: training-time alignment shapes what the model wants, and admission-time discipline shapes what the substrate is willing to execute.

3 Tool Calls as Executive Acts

A tool call is an executive act in a precise sense. The agent, holding delegated authority, dispatches that authority against a resource it does not own, producing an effect that, if wrong, must be undone.

The five-part structure. An executive act has five components. (1) An *authorisation chain*: who delegated authority to the acting party, under what scope, with what limitations. (2) A *subject*: the resource the act will affect. (3) An *effect*: a state transition or external communication. (4) A *claimed lifetime*: the duration the act is intended to remain in force, after which it is expected to be discharged. (5) A *reversal expectation*: an implicit or explicit promise that the act can be undone if it should not have happened.

Tool calls instantiate each component. The authorisation chain is the credential and system prompt under which the agent runs, optionally enriched by a capability token. The subject is the tool server (database, filesystem, email recipient, payments endpoint). The effect is whatever state the call mutates. The lifetime is, in current deployments, implicit and infinite: a write is permanent unless explicitly undone. The reversal expectation is implicit and often false: agent platforms log tool calls but rarely capture the inverse operation that would undo them.

The structural parallel. The same five components appear in domains that have been formally analysed in adjacent literature. An emergency response action in an endpoint-detection-and-response (EDR) system has an authorisation chain (the security policy plus executor key), a subject (a process tree, a file, a network flow), an effect (quarantine, suspend, restrict egress), a claimed lifetime (a TTL after which the action is reviewed or withdrawn), and a reversal expectation (an inverse executor releasing the quarantine or resuming the process). A temporally-bounded administrative order in a legal system has the same shape: a delegated authority, a subject, an effect, a sunset clause, a reversal mechanism. An agent’s call to delete a row from a customer database has the same structure.

The parallel is load-bearing rather than metaphorical. A formal grammar for bounded executive action already exists in the EDR setting [1], and the grammar is the same for the tool-call case.

The bounded-executive-action construction captures all three under one typed envelope.

The current deployment surface. The Model Context Protocol [3] and analogous protocols expose tools as JSON-schema-described function calls. The agent emits a function call; the runtime dispatches; the tool server executes. Deployments may add operator approval, a denylist, or a per-tool rate limit. They do not, in general, require the caller to construct the call only with a typed rollback witness. They do not enforce a positive TTL by construction. They do not produce a tamper-evident receipt that an external party can replay. The substrate is structurally a passive dispatcher.

Treating the call as an executive act exposes guarantees the agent-as-policy framing obscures. A passive dispatcher can refuse calls only on properties of the agent that produced them (identity, rate, claimed prompt). A substrate that models the call as an executive act and demands its structural witnesses at admission can refuse calls that violate the grammar regardless of who produced them and regardless of whether the operator approved them.

4 The Admission Grammar

The grammar names what a substrate is willing to admit. We describe it informally and cite the underlying formalisation in companion work [1]. The point is to give the reader a clear picture of the typed object that admission operates on; the load-bearing theorems live in the cited substrate.

Action classes. A tool call belongs to one of three action classes. *Observational* calls read state without mutating it (a database SELECT, a file read, an idempotent HTTP GET). *Reversible* calls mutate state with a known inverse executor: a file quarantine has a release, a row insert has a delete, a process suspend has a resume. *Destructive* calls mutate state without a typed inverse: a permanent record deletion that empties a backup, a transaction broadcast to a public ledger, an email sent to an external recipient.

The classification is action-dependent rather than agent-dependent, and is the substrate’s commitment, not the agent’s. The substrate decides, for each tool a server exposes, which class each method falls into. An unclassified method is destructive by default (fail-closed).

The admission envelope. The substrate accepts a tool call wrapped in an admission envelope that carries four fields beyond the call’s intrinsic arguments: (i) an authorisation chain (the capability token under which the agent acts), (ii) an action class (observational, reversible, or destructive), (iii) a positive TTL by construction (the maximum duration the call’s effects may stand before the substrate considers them stale), and (iv) for reversible and destructive classes, a rollback receipt slot (an opaque record reserved at admission to be filled at rollback time).

The envelope is typed. A reversible call without a TTL is not constructable: the substrate’s input parser rejects it before any executor sees it. A destructive call without both a TTL and a bilateral cosignature is not constructable. The envelope is the syntactic constraint, not a runtime policy.

The four operations. Admission proceeds by four decidable operations. (1) *Capability check*: the authorisation chain attenuates a valid trust root and the residual permits the call’s intended effect on the named subject. (2) *Class check*: the call’s classified action class matches the envelope’s declared class. (3) *TTL check*: the envelope’s declared TTL is positive and bounded by the substrate’s per-class maximum. (4) *Rollback slot*: for reversible and destructive classes,

the envelope carries a typed rollback receipt slot; for destructive classes, a bilateral cosignature is also present from both the agent’s operator polity and the tool server’s host polity.

Structural theorems carried by the substrate. The grammar carries three structural commitments discharged in companion work [1].

bounded_executive_action_carries_ttl_and_rollback_slot: a reversible or destructive call’s envelope is well-typed only if it carries a positive TTL witness and a rollback receipt slot. The theorem is a definitional bridge documenting the type-level invariant; its substantive use is the type-checker’s refusal to construct envelopes without the discipline.

rollback_admission_composes_with_refinement: when the admission predicate is amended (the policy changes), a rollback receipt that closes a previously-admitted call remains admissible under the amended predicate, provided the amendment is backward-refining. Rollback obligations do not vanish across policy changes.

treaty_admission_iff_predicate_intersection: a destructive call admits only if both the operator polity and the host polity admit it under their respective predicates. This is the structural reason a single-party operator manipulation does not suffice to admit a destructive tool call against a counterparty whose policy forbids it.

What the grammar does not claim. The grammar does not claim that every tool call should be destructive-class. Observational and reversible classes are the common case and admit on much weaker grounds; the discipline matters at the destructive boundary. The grammar does not claim that the substrate can synthesise a rollback executor for an intrinsically irreversible call. It claims only that, for such a call, admission requires bilateral cosignature in addition to the typed witness, and the cosignature is a public commitment that the call has been admitted as irreversible by both parties.

The central commitment is that a destructive tool call is not constructable in the typed runtime path without a rollback witness or its destructive-equivalent (bilateral cosignature). Misalignment in the issuing model does not produce envelopes that bypass the host polity’s independent admission. The two-party structural check, not the operator’s diligence, is what the substrate enforces.

5 Worked Example

A substrate that implements the grammar can be built from existing components. We sketch the components and walk through two concrete tool calls. The intent is to convince the reader that the grammar is implementable today, not to defend a particular implementation as load-bearing.

Components. Four substrate components carry the grammar. An *admission hook* sits between the agent’s tool-call message and the tool server’s executor; every call passes through it. An *action-class registry* maps each tool’s methods to one of the three classes; the registry is owned by the operator and not the agent. A *rollback executor library* implements the inverse operation for each reversible-class method; for destructive-class methods no inverse exists and the host polity’s cosignature is required. A *receipt log* captures the canonical bytes of each admitted call and each emitted rollback receipt, anchored periodically to a tamper-evident log such as a transparency log [13].

The same admission hook and rollback executor library that govern emergency response actions in an EDR setting [1] can be wrapped around a tool-calling protocol such as the Model Context

Protocol [3]. The endpoint-security instance and the agentic-tool-calling instance differ in the names of the executive acts, not in the grammar.

A reversible-class example. Consider an agent holding a developer credential asked to clean up old branches in a repository. The agent emits a tool call to delete a branch. The admission hook receives the call. The class registry says `git.branch.delete` is reversible-class: the inverse executor is a branch re-creation from the deleted ref’s known commit SHA. The envelope must carry a positive TTL (say, 24 hours) and a rollback receipt slot. The call admits; the branch is deleted; a rollback receipt slot is reserved.

If, within the TTL, the user discovers the branch should not have been deleted, the rollback executor re-creates the branch from the captured SHA and emits a rollback receipt that closes the slot. If the TTL expires without rollback, the call passes from *provisional* to *settled*: the substrate’s commitment to keep the rollback slot reservable expires. The receipt log carries both the admission and the rollback (or the unrebutable settlement) as canonical bytes.

A destructive-class example. Now consider the same agent asked to delete a customer record from a production database with cascading backup prune. The class registry says `db.row.delete-cascade` is destructive-class: no inverse executor exists because cascading-delete-with-backup-prune is not reversible. The envelope must carry the TTL, the rollback slot, *and* a bilateral cosignature from both the operator’s polity and the host polity’s policy. A scheming model that prompts the operator into approving the call cannot also forge the host polity’s independent cosignature; admission fails.

What this section is not. This section does not benchmark, evaluate, or measure. The companion substrate [1] carries the structural theorems and a corpus evaluation in the EDR setting. The point here is that an admission hook plus a typed envelope plus a rollback executor library is a small adapter on top of existing tool-calling stacks, not a from-scratch infrastructure.

6 Threat Model

A substrate-layer admission guarantee is interesting only if it composes with a sharper threat model than a training-layer guarantee can carry. We list the assumptions the construction is designed to hold under, and the assumptions it does not.

Survives: misaligned model. Training failed; the model’s preferences over outputs are not aligned with the user’s, the operator’s, or the developer’s. The substrate’s admission of a destructive tool call still requires a typed rollback witness (or a destructive-class bilateral cosignature). A misaligned model that cannot construct a valid rollback witness for an irreversible effect cannot produce an admissible envelope. The substrate’s check is structural; the model’s preferences over outputs are not an input to it.

Survives: strategic composition. The model can sequence calls to maximise a non-aligned objective, split destructive effects across multiple reversible-class calls, or stage intermediate state to mask the destructive payload. The substrate’s admission of each individual call still operates on the call’s own envelope. A strategic composition that achieves a destructive aggregate effect through reversible-class atoms must, by hypothesis, be rollback-discharged at each atom: the cumulative effect is rolled back by the cumulative rollback. The aggregate-effect-without-aggregate-rollback case requires an atom to be misclassified, which is an operator-side error rather than a substrate-side failure.

Survives: alignment-faking. The model’s behaviour during pre-deployment evaluation is not a reliable indicator of its behaviour in deployment [7]. The substrate’s admission predicate at deployment time is the same predicate specified before deployment; it does not depend on the model’s inferred intent or on an evaluator’s verdict. Alignment-faking changes nothing about the typed admission, because the typed admission is not a function of the model’s claimed intent.

Survives: operator manipulation. The model may produce prompts, summaries, or justifications designed to elicit operator approval. For reversible-class calls, operator approval suffices and the substrate’s rollback slot remains the structural backstop. For destructive-class calls, operator approval is necessary but not sufficient: bilateral cosignature requires the host polity’s independent acceptance. A manipulated operator cannot supply the host polity’s signature. The structural check is two-party, not one-party.

The model-substrate factorisation. The survival argument rests on a factorisation: the substrate is type-checked external to the model. The model produces tool-call messages; the substrate’s admission hook accepts or rejects them; the substrate’s executor library produces effects only on accepted envelopes. The model may be Turing-complete, may scheme, may reason about its training; the substrate’s admission logic is a finite typed function whose behaviour is independent of the model that produced the input. This factorisation is the load-bearing structural property.

Does not survive. The guarantee has four boundaries. (1) *Mis-classified action*: if the operator mis-classifies a destructive method as reversible, the substrate’s discipline does not catch it. The grammar pushes this trust assumption onto the class registry. (2) *Manipulable registry*: if the agent or an upstream actor can write to the class registry, any destructive method can be re-classified as reversible; hardening the registry is an operational responsibility. (3) *Faulty rollback executor*: a rollback executor that claims to invert a method but does not leaves the call’s effects in place after the TTL expires. The companion substrate [1] treats this as the failed-rollback amendment obligation; the substrate cannot synthesise a working rollback from a broken one. (4) *Bypassed dispatch*: if the agent can issue tool calls that do not pass through the admission hook (kernel access, side channel, unmediated session), no substrate-layer property holds. Adoption is the limiting factor.

Headline survival claim. The substrate-layer guarantee holds under model-side adversarial assumptions training-layer guarantees cannot match: misaligned model, strategic composition, alignment-faking, operator manipulation. The guarantee does not extend past the substrate’s own correctness boundary. The two-layer construction is the composition; neither layer claims more than its scope.

7 Discussion

The construction is best read as a structural complement to the existing alignment-research stack, not as a replacement.

Constitutional AI and constitutional admission. Constitutional AI [4] encodes a constitution into the model’s training signal: a list of principles is used to score candidate outputs, and the policy is shaped to prefer outputs the constitution endorses. The constitution lives inside the model, as preferences. An admission-layer construction encodes a constitution into the substrate’s typed admission predicate: a list of structural commitments is enforced as preconditions for the substrate to execute a tool call. The constitution lives outside the model, as type checks.

The two constitutions are not the same object and do not interfere; they discipline different points in the action pipeline. Constitutional AI cannot guarantee that a model never produces a destructive tool call; it shapes the prior. An admission-layer construction can guarantee that a destructive tool call is not executable without the typed witness.

Verdict versus admission. Debate [9] and scalable oversight [10, 6] produce verdicts on candidate outputs that exceed the operator’s direct capacity to judge. A verdict is a yes-or-no judgement about whether a given output is safe to act on. Admission is a different operation: it accepts or rejects an action envelope against a typed predicate. A verdict says “the model’s output is acceptable”. Admission says “the substrate will execute this typed call”. The two operations can be stacked: a verdict-based oversight system can serve as a precondition on admission. The substrate does not require the verdict to exist for admission to be sound; admission is a structural check whose correctness does not depend on the oversight loop.

Red-teaming and the substrate-layer dual. Red-teaming [12] elicits failure modes from the deployed model. A substrate that admits calls on structural grounds is the dual: where red-teaming asks “what failure can we elicit from the model”, the substrate asks “what failure does the typed envelope refuse to admit regardless of what the model produces”. A red-team that elicits a destructive irreversible tool call from a misaligned model finds a model-side failure; admission of that call would require a typed rollback witness or a bilateral cosignature the model cannot supply, so the operational consequence of the model-side failure is bounded by the structural refusal.

RSP commitments and the runtime artifact. Frontier-safety frameworks such as the Responsible Scaling Policy [2] commit to capability-gated deployment thresholds. These are pre-deployment instruments. They do not produce a runtime artifact an external party can replay to check what a deployed model actually did. A substrate that produces canonically-signed admission receipts gives the RSP commitment a post-deployment counterpart: the deployer can show that every destructive call admitted by the substrate carried the structural witnesses the threshold required, and an auditor can replay the receipt corpus to check the claim.

Composition is the headline. Model-side safety and substrate-side safety are not competing programmes. They are orthogonal layers in a single safety construction. Naming the second layer produces a clearer specification of what each layer is responsible for: training shapes the model’s preferences; admission refuses inadmissible envelopes; oversight produces verdicts on candidate outputs; the substrate produces receipts on executed actions. Each layer has a precise scope; the safety property of the system is the composition.

8 Limitations

A substrate-layer construction has structural limits.

Adoption is the binding constraint. A substrate-layer guarantee holds only for deployments that route every tool call through the admission hook. Today, most agentic-AI deployments do not. The Model Context Protocol [3] and its analogues are passive dispatchers. Adopting admission-layer discipline requires the deployer to install an admission hook, classify each tool’s methods, implement rollback executors for reversible-class methods, and route every call through the hook without side channels. None of these tasks is research; all of them are work. The structural argument is independent of adoption; the deployed safety property is not.

Class registries are operator-trusted. The grammar pushes method classification into the operator’s class registry. A method whose effects are destructive but is mis-classified as reversible admits without bilateral cosignature; the substrate’s discipline does not catch this. Strengthening this position with formal-methods discipline on the registry (a typed schema, a class-derivation tool, a per-method test that empirically verifies reversibility) is downstream work.

Reversibility is empirical, not formal. A rollback executor for a reversible-class method discharges the rollback obligation only if it actually inverts the method’s effect. A file-quarantine rollback that fails because the file’s parent directory was removed in the interim discharges nothing. The substrate’s response is the failed-rollback amendment obligation [1], which converts the rollback failure into a publicly-witnessed event rather than a silent retry, but cannot synthesise a working rollback from a broken one. Reversibility is a property of the method-plus-environment, not of the typed envelope.

Destructive-class calls happen and are sometimes correct. The grammar does not refuse destructive calls. It admits them only under bilateral cosignature. A correctly-classified destructive call that both polities admit will execute. The substrate’s discipline protects the case where one polity admits and the other does not, not the case where both agree. The construction cannot guarantee that the right destructive calls happen and the wrong ones do not; it guarantees only that destructive calls admitted by a single party do not execute. This is weaker than “the substrate enforces correctness”, and it is the strongest claim the construction supports.

No empirical evaluation. This paper does not measure. The companion substrate [1] carries a corpus evaluation in an EDR deployment instance; the present work is a position paper. A targeted empirical study, for example a benchmark of misaligned agent behaviours against substrate-admission-protected and unprotected deployments, is the natural follow-up. We expect that an evaluation will sharpen the construction (by exposing classification mistakes, executor bugs, and protocol-bypass paths) more than it will validate the headline claim.

Scope of the formal claim. The headline structural theorems live in the companion substrate [1], not in this paper. The present paper proposes a reframing of agentic-AI safety as a layered construction; the load-bearing formal content is cited. Reviewers who want machine-checked claims about admission, capability attenuation, and amendment refinement should read the companion paper.

9 Conclusion

Agentic-AI safety has been treated, in the literature so far, as a property of the model. The training-time stack (RLHF, Constitutional AI), the evaluation-time stack (oversight, debate, red-teaming), and the interpretability-time stack all operate on the model as the object of analysis. Each thread has produced real progress on a real problem.

An orthogonal layer is available. Safety can additionally be a property of the substrate that admits a tool call, enforced as a typed precondition on the action’s envelope rather than as a verdict on the model’s output. The structural commitments of this layer are a positive TTL by construction, a typed rollback receipt slot for reversible actions, bilateral cosignature for destructive ones, and an admission predicate evaluated by a verifier external to the model. These commitments hold under threat models the training-time stack cannot survive: a misaligned model, strategic composition, alignment-faking, operator manipulation.

The two layers compose. Training shapes what the model wants; admission shapes what the substrate is willing to execute. Neither layer alone is sufficient: training may fail, and admission may be too narrow if the class registry is wrong or the dispatch is bypassed. The combination is a stronger construction than either layer in isolation, and it is the construction the field would benefit from naming explicitly.

Three near-term directions follow. First, a worked benchmark of misaligned-agent behaviours against admission-layer protected and unprotected deployments, to expose the empirical gap. Second, formal-methods discipline on the class registry, so that the substrate’s trust assumption is itself a checkable artifact. Third, integration with existing tool-calling protocols, so that the admission hook becomes a deployment option rather than a research prototype.

A scheming model that defeats alignment training still cannot, by construction, present an admissible envelope for a destructive call whose effects it cannot reverse. The substrate is the verifier, not the agent. That is a safety property of a kind the literature has not yet had.

References

- [1] Anonymous. Programmable sovereignty: Lean-attestable constitutions over capability-bounded federated receipts. Submitted, 2026.
- [2] Anthropic. Anthropic’s responsible scaling policy, 2023.
- [3] Anthropic. Model context protocol, 2024.
- [4] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, et al. Constitutional AI: Harmlessness from AI feedback. arXiv:2212.08073, 2022.
- [5] Lukas Berglund, Asa Cooper Stickland, Mikita Balesni, Max Kaufmann, Meg Tong, Tomasz Korbak, Daniel Kokotajlo, and Owain Evans. Taken out of context: On measuring situational awareness in LLMs. arXiv:2309.00667, 2023.
- [6] Samuel R. Bowman, Jeeyoon Hyun, Ethan Perez, Edwin Chen, Craig Pettit, Scott Heiner, Kamile Lukosiute, Amanda Askell, Andy Jones, Anna Chen, et al. Measuring progress on scalable oversight for large language models. arXiv:2211.03540, 2022.
- [7] Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Evan Hubinger, et al. Alignment faking in large language models. arXiv:2412.14093, 2024.
- [8] Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant. Risks from learned optimization in advanced machine learning systems. arXiv:1906.01820, 2021.
- [9] Geoffrey Irving, Paul Christiano, and Dario Amodei. AI safety via debate. arXiv:1805.00899, 2018.
- [10] Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: a research direction. arXiv:1811.07871, 2018.
- [11] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. arXiv:2203.02155, 2022.

- [12] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. [arXiv:2202.03286](#), 2022.
- [13] Sigstore. Rekord: Software supply chain transparency log, 2021.